

MICROCONTROLLER

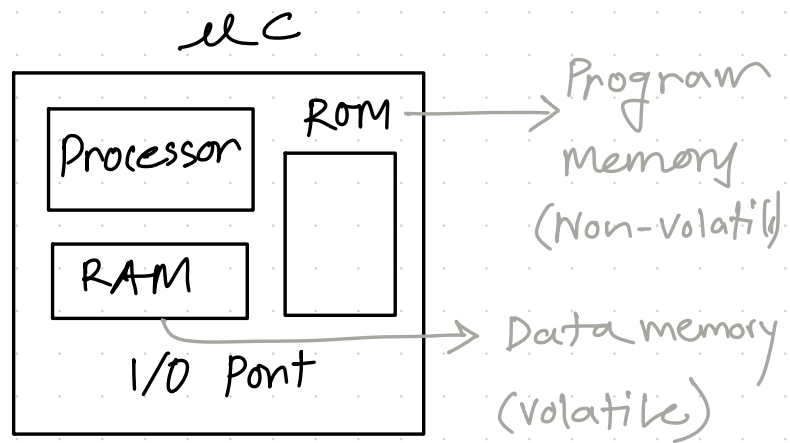
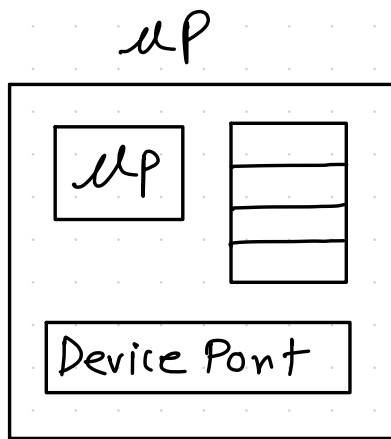
Microcontroller

Small

controls all components

μP Computer on chip (CoC)

μC System on chip (SoC)



Microprocessor vs Microcontroller

μP

- CPU is stand-alone, RAM, ROM, I/O are separate.
- Generally large in size.
- More expensive
- Higher power consumption.
- General purpose

μC

- CPU, RAM, ROM, I/O all are on a single chip.
- Small and compact in size.
- Cheaper than μP
- Lower power consumption.
- Built for specific/fixed purpose.

CLASSIFICATION OF μC

- ① Data Bus $\rightarrow$ 8 bit (8031)
 $\rightarrow$ 16 bit (8051)
 $\rightarrow$ 32 bit (8081)

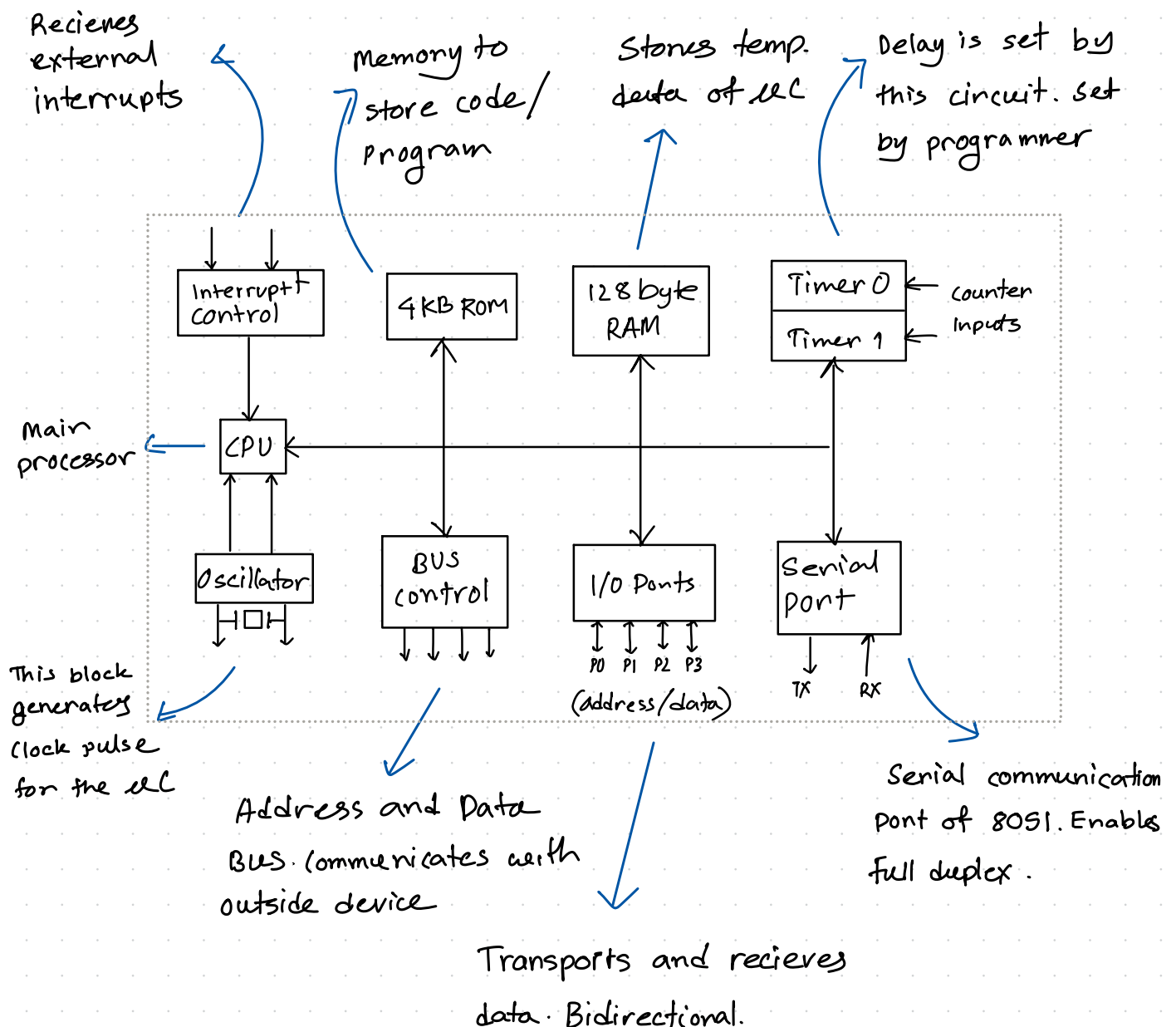
8051

ROM $\rightarrow$ 4 KB (4096)

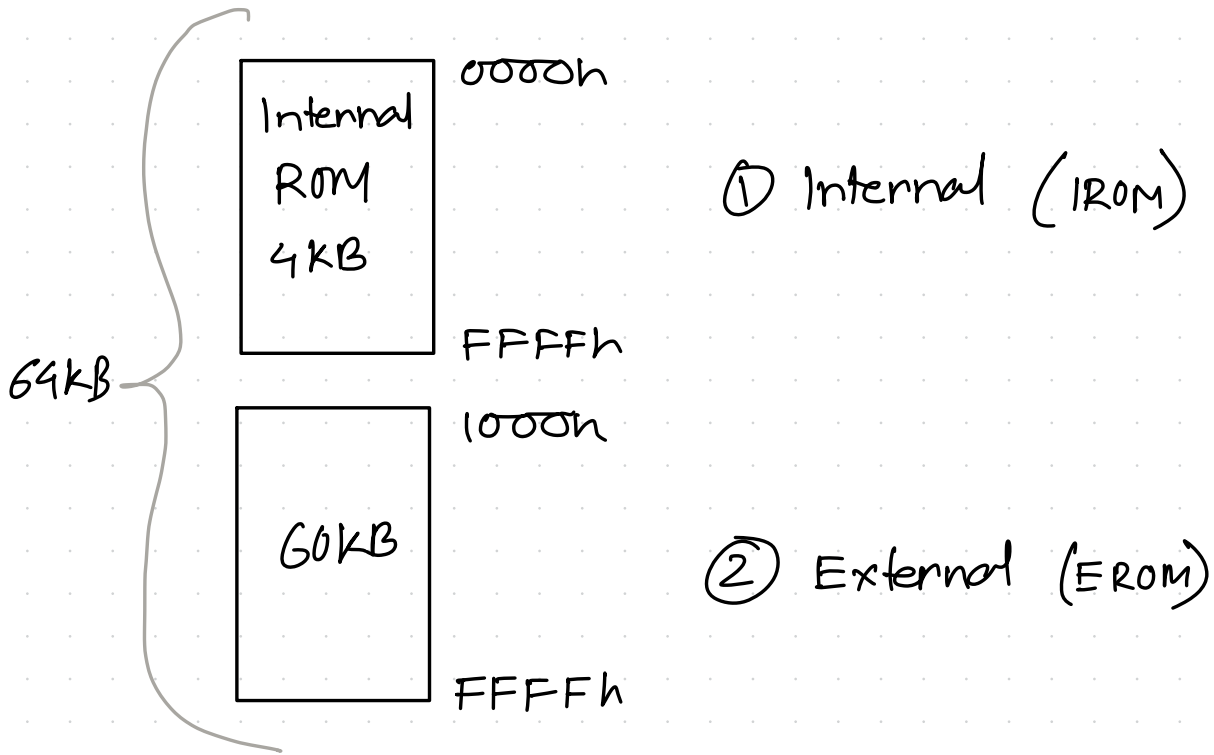
RAM $\rightarrow$ 128 bytes

- ② Internal memory $\rightarrow$ External (8031)
 $\rightarrow$ Embedded (8051)

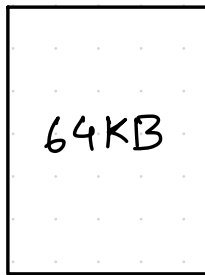
BLOCK DIAGRAM



ROM - READ ONLY MEMORY



③



If programmer wants a completely separate ROM to store a large program.

RAM - RANDOM ACCESS MEMORY

RAM — 128 byte
→ volatile

We can use external RAM if needed.

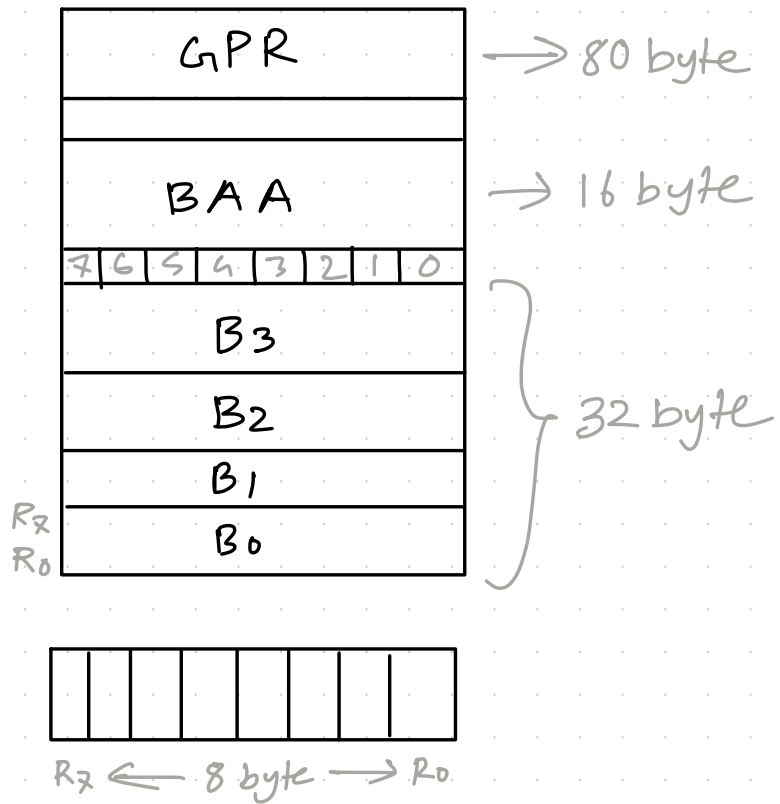
GPR → General Purpose Register

BAA → Bit Addressable Area

B₀-B₃ → Banks

* We can do bitwise operation in BAA

ROM vs RAM



ROM

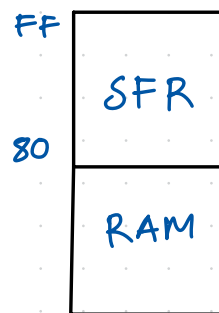
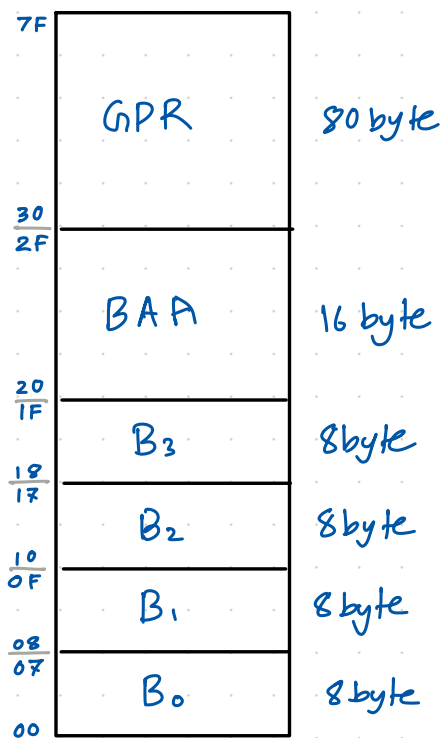
- Stores program
- Type: Non volatile
- 4kB ROM. Can be extended upto 64kB.
- $\overline{EA}$ pin works under ROM

RAM

- Stores Data
- Type: Volatile
- 128 byte RAM. Can be extended upto 64kB
- Not functional for RAM.

ROM $\rightarrow$ 4KB + ^{Extended} 60KB / 64K

RAM $\rightarrow$ 128 bytes (00-7F)



Special function
 $\rightarrow$ Register (128 byte)
 * sits on top of RAM

* SFR contains register like μP

- ① Accumulator (A & B)
- ② Status Register
- ③ Pointer register
- ④ Interrupt Reg
- ⑤ Stack pointer
- ⑥ I/O Register

① Accumulator (A & B) $8 + 8 = 16$ bit

$\rightarrow$ Performs math and logical operation

Ⓐ

8 bit

E0H

Ⓑ

8 bit

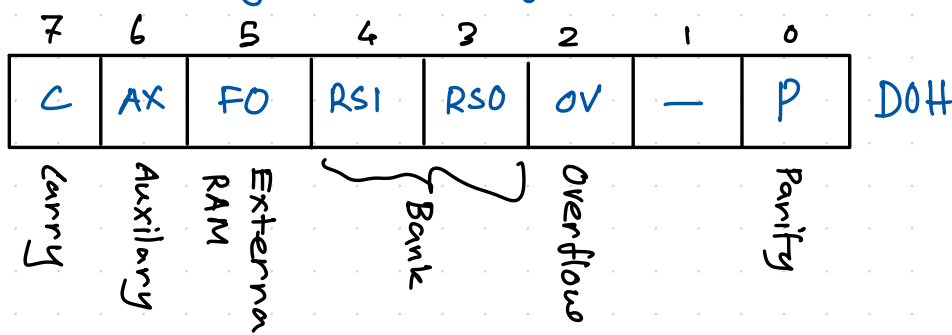
F0H

$\rightarrow$ size

$\rightarrow$ Memory location

$\rightarrow$ Mainly works A, involves B for division.

② Status Register (Flag)



RSI RSO

- 0 0 $\rightarrow$ B₀
- 0 1 $\rightarrow$ B₁
- 1 0 $\rightarrow$ B₂
- 1 1 $\rightarrow$ B₃

③ Pointer Register

→ To indicate CPU, which Bank we are using

DPH → Higher Bank (B_2, B_3) 83H

DPL → Lower Bank (B_0, B_1) 82H

④ Stack Pointer

⑤ I/O port register

P0 → 80H

P1 → 90H

P2 → A0H

P3 → B0H

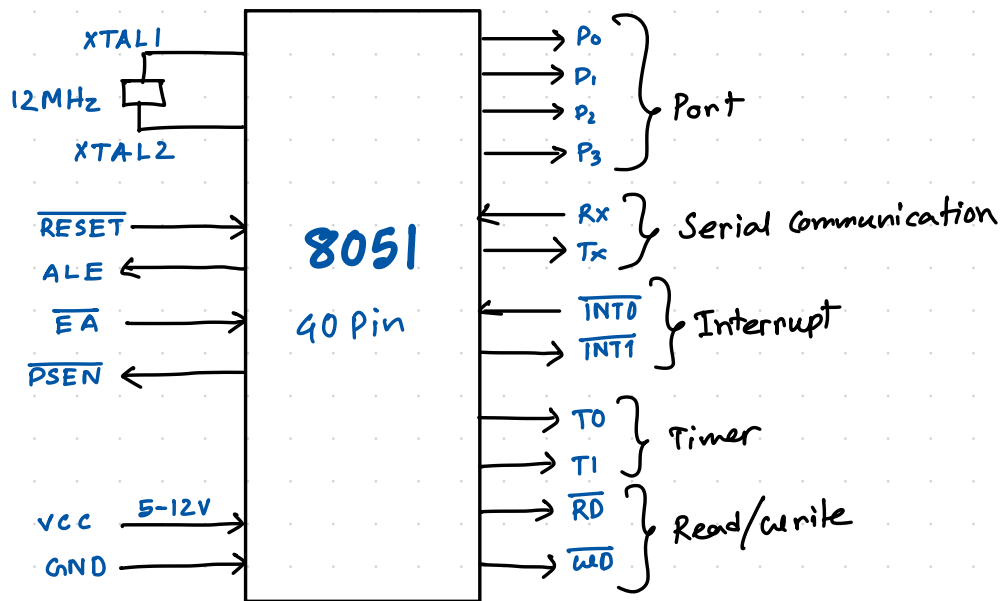
* 8 bit each

⑥ Interrupt Register

→ Primary Interrupt Register (PIR) A2H

→ Secondary " " (SIR) B2H

8051 PIN DIAGRAM



XTAL1-XTAL2 → Clock

Reset → Program Reset

ALE → Address Latch Enable

$\overline{EA}$ → Enable

→ 1 - Internal (4KB) ROM

→ 0 - Internal + External (4+60) KB ROM

$\overline{PSEN}$ → Program store enable

→ 1 - External RAM

→ 0 - No use

R_x → Data receive peripheral

T_x → Data send

"

Serial Communication

T₀, T₁ → Timer Circuit Control

INT₀ → Primary Interrupt

INT₁ → Secondary "

BIT MANIPULATION

Boolean Instructions

1. Set — `SETB 42H` → 1
 2. Clear — `CLR 42H` → 0
 3. Complement — `CPL 42H` → 0/1
- * Applicable for BAA (20H-2F)

Port 0, 1, 2, 3

8bit each

`SETB P0.2` → 1

↙
port location

Logical Operation

1. AND `ANL A, source`
2. OR `ORL A, source`
3. XOR `XRL A, source`
4. NOT `CPL destination`

`SWAP A`

`A = 78H`

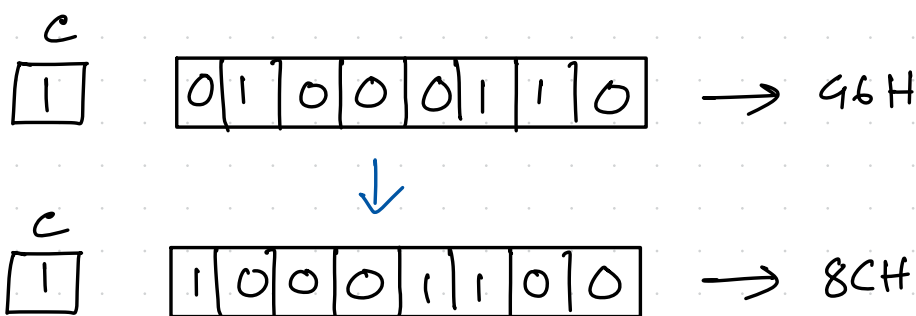
↓

`A = 87H`

Rotate

`RL destination/A`

`A = 46H`, Carry = 1



`RRC/RLC`
Rotates through
carry flag

`RR destination/A`

`ADD A, #17H` → Direct
`ADD A, 17H` → Memory
`ADD A, R2` → Bank
`ADD A, @R1` → ROM

`ADC` → Add with carry
`SUB` → Subtraction
`INC` → Increment
`DEC` → Decrement

ADDRESSING MODE

① Immediate Mode

MOV DPH, #82H

MOV A, #10H

MOV → Internal RAM only

We put # to indicate value, not address.

② Register mode

R0, R1, R2, R3 ... A, B, DPH, DPL → Registers

MOV R1, A [A will stay in memory, copy, not cut]

MOV R1, R2 ~~X~~ → Reg to Reg invalid

Instead we can do,

MOV 07H, R2 → Reg to memory

MOV R1, 07H → Memory to Reg

③ Direct Addressing

MOV A, 25H → Memory location of value.

MOV 25H, A to indicate value, we use #

MOV 15H, 25H

④ Indirect Addressing mode

To communicate with internal ROM (4KB) with RAM (128B)

MOV A, @R2 → Internal ROM R0, R1, R2, R3

MOVX A, R1 → External RAM to RAM
↘ External

Direct comm. from internal ROM to, External RAM is not possible. Instead

↘ Int ROM → Int RAM → Ex RAM

INTERRUPTS

↳ 6 Types

RESET

↳ RESET(1)

1st Priority

TO/TI

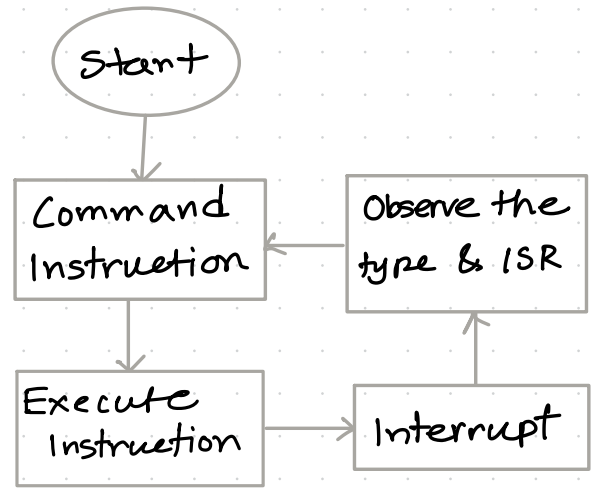
↳ Internal Interrupt (2)

INT0/INT1

↳ External " (2)

TI/RI

↳ Serial " (1)



Vector Address

Interrupt	Address
RESET	0000H
INT0	0003H
TO	000BH
INT1	0013H
TI	001BH
Serial	0023H

IE Register

7	6	5	4	3	2	1	0
EA	—	—	ES	ETI	EXI	ETO	EXO

✓

EA → Enable All $\begin{cases} 1-on \\ 0-off \end{cases}$

6/5 → Preserve bit (functional in 8096)

ES → Enable serial

ET →

IP Register

7	6	5	4	3	2	1	0
			PS	PTI	PXI	PTO	PXO

✓ H

Interrupt vs Polling **

* Briefly discuss how 8051 handles interrupts? (3a)

→

MEMORY INTERFACING

Chip Capacity $\rightarrow$ number of bits that a semi cond. can store
 $\rightarrow$ how many bits it can process (16bit in 8051)

Storage Capacity $\rightarrow$ how many bits of data it can store.

* Number of memory location depends on address pin.

* Number of bits each location can hold depends on data pin.

$2^x \rightarrow 12$ Address pin
 $x \rightarrow 4$ Data pin

$2^x \rightarrow$ No of Address pin $\rightarrow 2^{12} = 4096$ byte [4KB memory location]

$\rightarrow 4096 \times 4 = 16K$ [memory size]
 $\swarrow$
No of Data pin

4KB memory can hold 4bit in each location. Thus capacity 16K